

Veri Tabanı Yönetim Sistemleri II Ders notu

--- JOIN işlemleri ---

1. INNER JOIN

INNER JOIN, iki veya daha fazla tablodaki ortak alanlarda eşleşen kayıtları getirir. Bu komut, yalnızca her iki tabloda da bulunan (yani, eşleşen) verileri döndürür. Yani, bir tablodaki değer bir diğer tabloda bulunmuyorsa, o satır sonuçta yer almaz.

Kullanım Mantığı:

INNER JOIN, yalnızca "kesişim kümesi"ne odaklanır; yani, yalnızca her iki tabloda da bulunan satırlar sorgu sonucuna dahil edilir. Bu, ilişkili verilere dayalı birleştirmeleri sağlar.

Örnek:

İki tablomuz olsun: Musteriler ve Satışlar. Bu tablolarda, müşteri bilgileri ve yapılan satışlar yer almakta.

Örneğin, her iki tabloda da id alanı üzerinden birleştirme yapılabilir.

Musteriler Tablosu		Satışlar Tablosu		
id	Adi_soyadi	id	Satilan_mal	Satis_fiyati
1	Salih ESKİOĞLU	1	Buzdolabı	1200
2	Ayhan ÇETİNKAYA	1	LCD TV	1800
3	Serkan ÖZGÜREL	2	LCD TV	1750
4	İlhan ÖZLÜ	1	Çamaşır Makinesi	950

```
SELECT * FROM Musteriler
INNER JOIN Satışlar ON Musteriler.id = Satışlar.id;
```

Çıktı:

id	Adi_soyadi	Satilan_mal	Satis_fiyati
1	Salih ESKİOĞLU	Buzdolabı	1200
1	Salih ESKİOĞLU	LCD TV	1800
2	Ayhan ÇETİNKAYA	LCD TV	1750
1	Salih ESKİOĞLU	Çamaşır Makinesi	950

Bu komut ile, sadece her iki tabloda da yer alan müşteri ve satış bilgileri gösterilecektir. Eğer bir müşteri, satış tablosunda yer almıyorsa, o müşteri verisi sonuçlarda yer almaz.

Özetle:

- **Avantajı:** Gereksiz verilerden kaçınılır.
- **Dezavantajı:** Eşleşmeyen veriler atılır, dolayısıyla eksik bilgi elde edilebilir.

2. LEFT JOIN (veya LEFT OUTER JOIN)

LEFT JOIN, sol tablodaki tüm verileri alır ve sağ tablodaki eşleşen verileri döndürür. Sağ tablodaki eşleşmeyen veriler için NULL döndürülür. Bu, sol tablodaki her kayıt için, sağ tablodan mümkünse bir eşleşme getirir. Eğer sağ tabloda eşleşen bir kayıt yoksa, yine sol tablodaki tüm veriler döndürülür, ancak sağ tablodan gelen bilgiler NULL olur.

Kullanım Mantığı:

Sol tablodaki verilerin kaybolmamasını sağlayan JOIN türüdür. Eğer sağ tablodan eşleşen bir satır yoksa, NULL değeri döndürülür.

Örnek: Aynı kitaplar ve yazarlar tablosuyla örnek verecek olursak, tüm kitaplar ve eşleşen yazarları almak için LEFT JOIN kullanılabilir:

• kitaplar Tablosu						
id	kitap_isim	Yıl	sayfa	Fiyat	yazarid	yayinevid
1	saklı ev	1995	255	20	1	2
2	son moda	2002	652	25	null	2
3	gelecek	2007	214	12	2	1
4	bir umut	2008	421	18	3	1

• yazarlar Tablosu			
id	yazar_isim	soyisim	yas
1	ekrem	sever	28
2	orhan	erdem	24
3	orhan	karasu	34
4	isa	bereket	43

```
SELECT kitaplar.kitap_isim, yazarlar.yazar_isim
FROM kitaplar
LEFT JOIN yazarlar ON kitaplar.yazarid = yazarlar.id;
```

çıktı:

kitap_isim	yazar_isim
saklı ev	ekrem
son moda	null
gelecek	orhan
bir umut	orhan

- LEFT JOIN soldaki tablonun tamamını alır ve diğer tablodaki ilişkili verileri döndürür.

Eğer kitaplar tablosundaki bir kitap için yazar bilgisi yoksa, yine kitap adı döner ancak yazar_isim NULL olur.

Özetle:

- **Avantajı:** Sol tablodaki verilerin kaybolmaması sağlanır.
- **Dezavantajı:** Sağ tabloda eşleşmeyen verilerde NULL döndürülmesi, veri eksikliğine yol açabilir.

3. RIGHT JOIN (veya RIGHT OUTER JOIN)

RIGHT JOIN, LEFT JOIN ile hemen hemen aynıdır, ancak bu sefer sağ tablodaki tüm kayıtları döndürür. Eğer sol tabloda eşleşen bir kayıt yoksa, sol tablodan gelen değerler NULL olarak döner.

Kullanım Mantığı:

Sağ tablodaki tüm veriler gösterilir. Sol tablodan yalnızca sağ tabloyla eşleşenler gelir. Sağ tablodan eşleşmeyen kayıtlar yine NULL olarak gösterilir.

Örnek: kitaplar ve yazarlar diye iki tablomuz mevcut

• kitaplar Tablosu						
id	kitap_isim	yl	sayfa	fiyat	yazarid	yayineviid
1	saklı ev	1995	255	20	1	2
2	son moda	2002	652	25	null	2
3	gelecek	2007	214	12	2	1
4	bir umut	2008	421	18	3	1

• yazarlar Tablosu			
id	yazar_isim	soyisim	yas
1	ekrem	sever	28
2	orhan	erdem	24
3	orhan	karasu	34
4	isa	bereket	43

```
SELECT kitaplar.kitap_isim, yazarlar.yazar_isim
FROM kitaplar
RIGHT JOIN yazarlar ON kitaplar.yazarid = yazarlar.id;
```

çıktı:

kitap_isim	yazar_isim
saklı ev	ekrem
son moda	orhan
gelecek	orhan
bir umut	orhan
NULL	isa

- Görüldüğü gibi sağda yer alan yazarlar tablosunun tamamını sonuç olarak aldı ve yazara karşılık gelen bir kitap bulunmadıysa ilgili alanın değerini “null” olarak atadı.

Burada, tüm yazarlar listelenir ve her yazarın kitabı varsa, kitap adı da döndürülür. Eğer bir yazarın kitabı yoksa, kitap adı NULL olur.

Özetle:

- **Avantajı:** Sağ tablodaki verilerin kaybolmaması sağlanır.
- **Dezavantajı:** Sol tabloda eşleşmeyen veriler NULL olur.

4. FULL JOIN (veya FULL OUTER JOIN)

FULL OUTER JOIN, SQL'de iki tablonun tüm verilerini birleştirirken hem sağdaki hem de soldaki tablodan gelen tüm kayıtları alır. Eğer sağdaki veya soldaki tablodan eşleşmeyen bir kayıt varsa, o satırda ilgili tablo için NULL değeri döner.

Kullanım Mantığı:

- **Her iki tablodan da tüm veriler gösterilir:** Eşleşen kayıtlar birleştirilir, ancak eşleşmeyen kayıtlar için diğer tablodan NULL değerleri gösterilir.
- **Sağdaki ve soldaki tablodan veriler:** Eşleşmeyen satırlar NULL değerleri ile gösterili

Örnek: kitaplar ve yazarlar diye iki tablomuz mevcut

	id	kitap_isim	yil	sayfa	fiyat	yazarid	yayinvid
1	1	saklı ev	1995	255	20.00	2	1
2	2	son moda	2002	652	25.00	NULL	2
3	3	gelecek	2007	412	14.00	3	1
4	4	bir umut	2012	365	18.00	1	3
5	5	Dijital Kale	1998	496	30.00	4	1
6	6	Yüzüklerin Efendisi	1954	1200	75.00	5	2

	id	yazar_isim	soyisim	yas
1	1	orhan	sever	28
2	2	orhan	erdem	24
3	3	orhan	karasu	34
4	4	isa	bereket	43
5	7	Ahmet	Kaya	40
6	8	Mehmet	Yılmaz	32

SELECT * FROM kitaplar FULL JOIN OUTER yazarlar

ON kitaplar.id = yazarlar.id;

çıktı:

	id	kitap_isim	yil	sayfa	fiyat	yazarid	yayinvid	id	yazar_isim	soyisim	yas
1	1	sahik ev	1995	255	20.00	2	1	1	orhan	sever	28
2	2	son moda	2002	652	25.00	NULL	2	2	orhan	erdem	24
3	3	gelecek	2007	412	14.00	3	1	3	orhan	karasu	34
4	4	bir umut	2012	365	18.00	1	3	4	isa	bereket	43
5	5	Dijital Kale	1998	496	30.00	4	1	NULL	NULL	NULL	NULL
6	6	Yüzüklerin Efendisi	1954	1200	75.00	5	2	NULL	NULL	NULL	NULL
7	NULL	NULL	NULL	NULL	NULL	NULL	NULL	7	Ahmet	Kaya	40
8	NULL	NULL	NULL	NULL	NULL	NULL	NULL	8	Mehmet	Yılmaz	32

FULL OUTER JOIN komutuyla yapılan sorgu sonucunda, her iki tablodaki tüm veriler alınmıştır. Görüldüğü gibi, sağda yer alan **yazarlar** tablosunun tamamı sonuç olarak alınmıştır ve bir yazara karşılık gelen bir kitap bulunmadığı takdirde, o yazara ait tüm kitap bilgileri **NULL** olarak atanmıştır.

Sonuçta, **FULL OUTER JOIN** kullanıldığında, her iki tablonun tüm verilerini almak mümkün olur. Eşleşmeyen kayıtlar **NULL** olarak döndürülür.

5. UNION

UNION, iki veya daha fazla SELECT komutunun sonuçlarını birleştirir. Ancak, birleştirilen sonuçlar tekrarsız olmalıdır. Yani, aynı veriler yalnızca bir kez döndürülür. UNION ALL komutuyla ise tekrar eden veriler de dahil edilir.

Kullanım Mantığı:

Birleştirilen SELECT komutlarının her birinin aynı sayıda sütun içermesi ve her sütunun aynı veri tipinde olması gerekir.

Örnek:

Personel ve Musteriler tablosundaki şehir verilerini birleştirelim:

• Personel tablosu		
id	Adi_soyadi	Sehir
1	Salih ESKİOĞLU	İstanbul
2	Ayhan ÇETİNKAYA	Kocaeli
3	Serkan ÖZGÜREL	Erzincan
4	İlhan ÖZLÜ	İstanbul

Musteriler tablosu		
id	Adi_soyadi	Sehir
1	Veysi Yamli	Van
2	Sırrı Derman	Mersin

```
SELECT Sehir FROM Personel
UNION
SELECT Sehir FROM Musteriler;
```

çıktı:

Sehir
İstanbul
Kocaeli
Erzincan
Van
Mersin

Bu komut ile, her iki tablodaki şehir bilgileri birleştirilir ve tekrarlanan şehirler sadece bir kez yer alır.

Özetle:

- **Avantajı:** Farklı tablolardan gelen veriler tek bir sonuç kümesinde birleştirilir.
- **Dezavantajı:** Tekrar eden veriler yok sayılır (veya UNION ALL kullanıldığında tam tersi yapılır).

Özetle JOIN Türleri Arasındaki Farklar:

- **INNER JOIN:** Sadece eşleşen veriler döndürülür. Her iki tabloda da bulunan veri alınır.
- **LEFT JOIN:** Sol tablodaki tüm veriler alınır, sağ tablodan eşleşenler varsa getirilir. Sağ tablodan eşleşmeyenler NULL olur.
- **RIGHT JOIN:** Sağ tablodaki tüm veriler alınır, sol tablodan eşleşenler varsa getirilir. Sol tablodan eşleşmeyenler NULL olur.
- **FULL OUTER JOIN:** Her iki tablodan da tüm veriler döndürülür. Eşleşen kayıtlar birleştirilir, eşleşmeyenler NULL olarak döner.
- **UNION:** Birleştirilen iki veya daha fazla **SELECT** sorgusunun sonuçları döndürülür, ancak tekrarlanan satırlar yalnızca bir kez alınır.

--- STRING-FONKSİYONLARI ---

Bu bölümde, SQL'deki bazı **string fonksiyonları** bulunmakta. İşte bu fonksiyonlar:

1. **ASCII**: Bir karakterin ASCII kodunu döndürür. Örneğin, 'A' karakterinin ASCII değeri 65'tir.
2. **CHAR**: Bir sayıyı karaktere dönüştürür. Örneğin, 65 sayısı 'A' karakterine dönüşür.
3. **SUBSTRING**: Bir dizgeden belirli bir bölüm alır.
4. **REPLACE**: Bir dizgedeki belirli bir metni başka bir metinle değiştirir.
5. **CHARINDEX**: Bir dizgedeki belirli bir alt dizgenin konumunu bulur.
6. **CONCAT**: İki veya daha fazla dizgeyi birleştirir.
7. **CONCAT_WS**: CONCAT ile benzer, ancak bir ayırıcı karakter (delimiter) kullanarak dizgeleri birleştirir.
8. **FORMAT**: Verilen değeri belirtilen formata dönüştürür.
9. **RTRIM**: Sağ taraftaki boşlukları temizler.
10. **LTRIM**: Sol taraftaki boşlukları temizler.
11. **TRIM**: Dizgenin başındaki ve sonundaki boşlukları temizler.
12. **LEFT**: Bir dizgenin solundan belirtilen sayıda karakter alır.
13. **RIGHT**: Bir dizgenin sağından belirtilen sayıda karakter alır.
14. **LEN**: Dizgenin uzunluğunu döndürür.
15. **LOWER**: Dizgedeki tüm karakterleri küçük harfe dönüştürür.
16. **UPPER**: Dizgedeki tüm karakterleri büyük harfe dönüştürür.
17. **REVERSE**: Bir dizgeyi tersine çevirir.
18. **REPLICATE**: Bir dizgiyi belirtilen sayıda tekrarlar.

Bu fonksiyonlar genellikle metin verisi üzerinde çeşitli manipülasyonlar yapmak için kullanılır ve SQL'deki veri işleme işlevselliğini artırır.

ÖRNEKLER:

Örnek:

```
SELECT ASCII ('A')
```

Çıktı:

65

- 'A' karakterinin ASCII değeri 65'tir.

Örnek:

```
SELECT CHAR (47)
```

Çıktı: /

- 47 sayısı '/' karakterine dönüşür.

Örnek:

```
SELECT SUBSTRING ('İSTANBUL BELEDİYESİ',1,4)
```

Çıktı: İSTA

- içerdeki yazının 1 den 4 kadar olan karakterleri alınmıştır

Örnek:

```
SELECT CHARINDEX ('N','KENT ÜNİVERSİTESİ')
```

Çıktı: 3

- Bu örnekte 'N' harfi 'KENT ÜNİVERSİTESİ' metninde 3. pozisyonda bulunuyor, dolayısıyla 3 çıktısını alıyoruz.

Örnek:

```
SELECT CONCAT ('İSTANBUL',' ','KENT',' ','ÜNİVERSİTESİ')
```

Çıktı: İSTANBUL KENT ÜNİVERSİTESİ

- ' ' (tek tırnak içinde boşluk) bir boşluk karakteridir ve bu, metinler arasına eklenir.
- Bu örnekte: 'İSTANBUL', ' ', 'KENT', ' ', ve 'ÜNİVERSİTESİ' dizgeleri birleştiriliyor.

Örnek:

```
SELECT CONCAT_WS(' ','KAĞITHANE','KENT','ÜNİVERSİTESİ')
```

Çıktı: KAĞITHANE KENT ÜNİVERSİTESİ

- İlk parametre, ayırıcıdır (bu örnekte ' ' boşluk karakteri).
- Geriye kalan parametreler ise birleştirilecek dizgelerdir ('KAĞITHANE', 'KENT', 'ÜNİVERSİTESİ').
- ' ' (boşluk) ayırıcı olarak kullanılır. ' ' (boşluk) karakterini en başa koyduğumuz için **CONCAT** komutunun aksine her dizgi arasına boşluk koymamıza gerek kalmaz. **CONCAT_WS** komutunda bunu en başta belirtmiş oluyoruz.

Örnek:

```
SELECT GETDATE()
```

Çıktı: 2025-04-10 05:31:40.403

```
SELECT FORMAT(GETDATE(),'D','en-us') as ukformat
```


Çıktı: Thursday, April 10, 2025

- **FORMAT()** fonksiyonu, tarih veya saat verisini belirli bir formata dönüştürmek için kullanılır.
- Bu sorgu çalıştırıldığında, geçerli tarih şu şekilde döner:
Thursday, April 10, 2025
- Bu kısımda **FORMAT** komutuna odaklanmamız gerekiyor
- **GETDATE()**: Geçerli tarih ve saat bilgisini döndürür.
- **FORMAT()**: Tarih bilgisini istenilen formatta döndürür. Burada '**D**' kullanımı, günü, ayı ve yılı uzun formatta gösterir.

Örnk:

SELECT LEFT ('ÜNİVERSİTE',4)

Çıktı: ÜNİV

- En Soldan 4 karakter alır

SELECT RIGHT ('ÜNİVERSİTE',2)

Çıktı: TE

- En Sağdan 2 karakter alır

Örnk:

SELECT LEN ('ÜNİVERSİTE')

Çıktı: 10

- **LEN()** fonksiyonu, verilen dizgenin **karakter sayısını** döndürür.
- Bu örnekte, 'ÜNİVERSİTE' kelimesinin 10 karakterden oluştuğu için sonuç **10** olarak döner.

Örnk:

SELECT UPPER('lionel messi') -----> çıktı: LİONEL MESSİ

- Dizideki karakterleri büyüttü

SELECT LOWER('LİONEL MESSİ') -----> çıktı: lionel messi

- Dizideki karakterleri küçülttü

SELECT REVERSE('KENT ÜNİVERSİTESİ') -----> çıktı: İSETİSREVİNÖ TNEK

- Diziyi tersine çevirdi

SELECT REPLICATE ('1',10) -----> çıktı: 1111111111

- Dizi içerisindeki 1 sayısını 10 defa tekrarlayarak ekrana getirdi

--- Stored Procedures ve SQL Transaction Yönetimi ---

Bu bölümde, **Stored Procedure** (Saklı Yordam) ve **SQL Transaction** (İşlem) konularını ele alacağız. İşte her iki terim hakkında detaylı bilgiler:

Stored Procedure (Saklı Yordam)

- **Tanım:** SQL veritabanlarında, önceden yazılmış ve saklanmış SQL komutlarıdır. Tekrar tekrar çalıştırılması gereken sorgular için kullanılır.
- **Özellikler:**
 - **Performansı artırır:** Tekrar edilen işlemleri derleyip saklayarak veritabanı sunucusunda tutar.
 - **Kod tekrarını azaltır:** Aynı SQL komutlarını her seferinde yazmak yerine, sadece saklı yordamı çağırabilirsiniz.
 - **Güvenliği artırır:** Kullanıcıların doğrudan tablo üzerinde işlem yapmasını engeller ve veri güvenliğini sağlar.

SQL Transaction (İşlem)

- **Tanım:** Bir veya daha fazla SQL sorgusunun bir bütün olarak çalışmasını sağlar. İşlemler ya başarılı olur ve kalıcı hale gelir, ya da bir hata durumunda tüm işlem geri alınarak (ROLLBACK) sistemin tutarlılığı sağlanır.
- **Özellikler:**
 - **ACID ilkelerine uyar:**
 - **Atomicity:** Tüm işlemler ya tamamlanır ya da hiçbiri yapılmaz.
 - **Consistency:** Veritabanı her zaman tutarlı bir durumda kalır.
 - **Isolation:** Paralel işlemler birbirini etkilemez.
 - **Durability:** İşlem tamamlandıktan sonra veriler kalıcı olarak saklanır.

Transaction Kullanımı:

Genellikle **INSERT**, **UPDATE** ve **DELETE** işlemleri için kullanılır. Örneğin, veritabanına

veri eklerken veya güncellerken, tüm işlem başarılı olursa veriler kaydedilir. Eğer bir hata olursa, tüm işlem geri alınır.